PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.9

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

II Representing Pattern 13

Uncertainty Estimation 15

III Epistemic Pattern 23

Transfer Learning 25

IV Aleatoric Pattern 45

Index 47

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Part II

Representing Pattern

Uncertainty Estimation

I know that I know nothing.

The Why

In the previous chapter we learned to compress high-dimensional data into structured latent spaces and to draw new samples from those spaces. The Variational Autoencoder mapped each input to an embedding, decoded it back, and could even synthesise plausible new samples from the prior. Neither capability told us how much we should trust those outputs for any particular input. A reconstruction looks the same whether the input lay in the heart of the training distribution or far from any encoded neighbour in feature space; the model has no built-in way to communicate its own uncertainty.

Neural networks are trained to predict, and they do so with impressive accuracy on familiar data, which makes them particularly misleading on unfamiliar data.

Models trained on finite data are confident outside their training distribution by default. The softmax of a classifier sums to one regardless of whether the input is a familiar digit or a sample drawn from pure noise. The decoder of a VAE produces a smooth output regardless of whether the latent code corresponds to a region the encoder ever visited. Confidence is a number the model returns; it is not a flag that the model has actually seen anything like this input before, nor a measure of how far the input sits from the embeddings of the training data.

The long tail. Real-world data is heavy-tailed: a small number of common situations cover most of the volume, but a long tail of rare configurations refuses to vanish no matter how much data is collected. Autonomous vehicles see most of the road most of the time and meet the unusual every few hours, and a model that is confident on the head will be confident on the tail by default.

P. Koopman. The heavy tail safety ceiling. In *Automated and Connected Vehicle Systems Testing Symposium*, 2018. https://users.ece.cmu.edu/~koopman/pubs/koopman18_heavy_tail_ceiling.pdf

Silent failure is the natural consequence. A model that cannot express doubt will eventually be confidently wrong, and no downstream safeguard can compensate for an uncertainty signal that was never produced in the first place. The classifier returns its prediction, the planner trusts it, the actuator executes; the loop closes around the wrong answer with no visible alarm.

Acting under uncertainty is what turns a raw doubt signal into

a deployable safety net. Three operational layers compose to keep the loop open. Human-in-the-loop puts a reviewer on the critical path: the model emits a prediction with an uncertainty score, and the system stalls until the reviewer accepts or overrides. Human-on-the-loop runs the model autonomously and escalates only flagged cases, so the reviewer steers without blocking the average prediction. Fail-safe defaults are the conservative action taken when the model has flagged the input as outside its competence and no reviewer is available. Behind all three is monitoring, the slow loop that watches the distribution of scores over time and alerts when the input world has drifted from the world the model was trained on. A complementary thread is explainability[1]: when a flagged decision is shown to a reviewer, surfacing the features or training neighbours that drove the prediction tells them where to look, so calibration tells the reviewer when to look and explanation tells them what to look at.

[1] M. T. Ribeiro, S. Singh, and C. Guestrin. "Why Should I Trust You?": Explaining the predictions of any classifier. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. https://arxiv.org/abs/1602.04938

Confidence is not correctness. Neural networks calibrate poorly on out-of-distribution inputs by design: the softmax normalises to sum to one whether the input is a training-set image or a frame from a film the network has never seen. Adversarial inputs lift the problem to a sharper edge, where the model is high-confidence wrong on imperceptible perturbations. The cure is not to make the model more confident; it is to make the model honest about what it does not know, so that confidence and accuracy track each other on a calibration plot rather than drifting apart. A complementary route is to train a small head whose only job is to predict whether the main model will be right on the current input, turning the model's own likelihood of being correct into a learned routing signal that sends the cases it is likely to mishandle to a reviewer before they cause damage.

Calibration plot. Bin predictions by stated confidence, then plot mean confidence on the horizontal axis against empirical accuracy on the vertical. The diagonal is perfect calibration; modern deep classifiers typically sit above the diagonal, which is to say they are systematically overconfident.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017. https://arxiv.org/abs/1706.04599

Different sources of uncertainty call for different responses. Some uncertainty is irreducible noise in the data and will not shrink with any amount of training. Some uncertainty reflects gaps in what the model has been shown and shrinks the moment relevant data arrives. Treating the two the same wastes effort and conceals the actionable signal under the inevitable one.

Uncertainty estimation closes the gap. It asks: given a model trained on finite data under specific assumptions, how reliable are its outputs for a particular input, and what kind of unreliability is at play? The answer turns a single output into a distribution over outputs, a single confidence number into a structured signal that downstream code can route on. A close cousin of the same problem

ODD and drift. The operational design domain names the conditions under which a system was designed to work, namely the slice of the world the training data covered. ODD detection asks at runtime whether the current input falls inside that slice. Data drift is the slow change in input statistics over time; model drift is the resulting decoupling of stated confidence from empirical accuracy. Both bend the calibration plot away from the diagonal long before the operator notices.

is anomaly detection trained as one-class learning: deliberately fit only normal data, then flag any input the model finds unusual without ever showing it a labelled anomaly. The same uncertainty signal that powers calibration also powers detection of inputs the training distribution never covered, which is the regime where labels are rare and the events that matter most are the ones nobody has annotated.

In order to move from confident predictions to calibrated ones we have good reason to find ways to,

- separate irreducible data noise from reducible model ignorance, so that each source of uncertainty triggers the right response.
- quantify how much a model's predictions vary across plausible parameter settings, turning a single output into a distribution over outputs.
- act on uncertainty estimates in practice, routing unreliable predictions to human reviewers, falling back to fail-safe defaults, and detecting inputs the model was never trained to handle.

Hands On Experience

Step into the role of the classifier. Imagine you have just finished a clustering pass over a training set and the result you walk away with is binding: there are exactly two classes in the world, the cluster on the left and the cluster on the right. We will hand you grey candidates one at a time and your job is to commit each one to either left or right. You are not allowed to abstain, you are not allowed to invent a third option, and you have to give an answer for every point we show you. Read Figure 1 as the world the rules describe: ten training points define what a class looks like, and four grey candidates wait for your verdict.

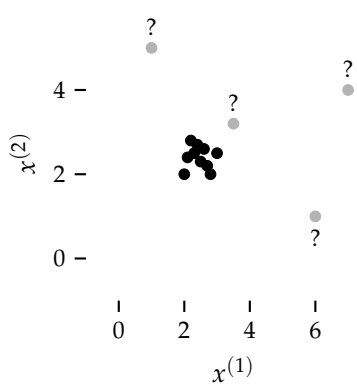

Figure 1: Ten training points clustered around $(2.5, 2.5)$ in black define the only two classes the rules permit; four grey candidates sit outside that cluster at varying distances and directions. The candidate at $(3.5, 3.2)$ is an easy commit, the candidates at $(6, 1)$ and $(1, 5)$ are borderline in opposite directions, and the candidate at $(7, 4)$ sits far enough out that the rules feel wrong.

The first candidate is easy. The grey point at $(3.5, 3.2)$ sits a hair outside the dense cloud, in the direction the cluster is already trending. You commit it to the same class without much hesitation, and your internal confidence is high. This is the regime classifiers are built for: the input looks like the training data, the answer is the obvious one, and there is no friction between the rules and your intuition.

The borderline pair is uncomfortable. The candidate at $(6, 1)$ sits well to the right, away from anything you have seen, and the candidate at $(1, 5)$ sits well above the cluster in a different direction entirely. The rules still demand a single class, so you commit, but the commitment now feels arbitrary: nothing in the data tells you these two grey points belong together, only the impoverished label set does. Notice that you have just produced two predictions whose internal confidence has dropped sharply, and yet a softmax over two classes will normalise to one regardless and give you no place to record that drop.

The far outlier breaks the rules. The candidate at $(7, 4)$ sits far enough from either training neighbourhood that the obvious answer is none of the above, and at this point we tell you what you already suspected: this point is a new class emerging in the wild, a regime your training pass never covered. Your earlier commitment was forced and wrong, but the wrongness was never visible in the prediction itself, only in the gap between the input and everything you had ever seen. This is the silent-failure mode from the section above made tactile: the loop closed around your answer and there was no flag to open it back up.

Open questions for the role-play.

- Which candidate did you commit with the most discomfort, and what would you have written next to that prediction if you had been allowed?
- If the points at $(6, 1)$ and $(1, 5)$ both became their own new classes tomorrow, what would change in your earlier commitments?
- How would you, as a system designer, prevent a downstream actuator from acting on the $(7, 4)$ commitment with the same trust as the $(3.5, 3.2)$ one?

Now for the second twist. We come back and reveal that the ten training points were never one class at all: the lower band around $y \approx 2$ and the upper band around $y \approx 2.7$ have been two distinct subclasses since the beginning, and the clustering pass simply failed to resolve them. Your earlier commitment to $(3.5, 3.2)$ now needs to split into two different answers depending on which subclass it should belong to, except you no longer have access to that earlier prediction in a form you can re-route. The same trap shows up in deployment as slow drift: the world your model was trained on was already richer than your label set admitted, and the bill comes due once the richness becomes operationally relevant.

Step out of the role and design the missing pieces. What the exercise made absent is exactly what an uncertainty-aware sys-

tem has to provide. First, an I-am-not-sure flag the model can raise when its commitment is forced rather than confident, so the borderline pair never travels downstream with the same trust as the easy commit. Second, a routing rule that sends flagged inputs to a human reviewer who can override, defer, or label them as something new, turning the human-in-the-loop layer of the section above into a concrete pipeline stage. Third, a novelty detector that watches for inputs unlike anything in the training distribution, so the $(7, 4)$ kind of point is recognised as new before it is forced into an existing class. Fourth, a drift monitor that tracks the population of inputs and predictions over time, so that a slow subclass split is caught while it is still a small distortion of the calibration plot rather than a system-wide breakdown.

UNCERTAINTY ESTIMATION SURFACES THE GAP. The same prediction now comes with a measure of doubt that downstream code can act on. Instead of one number we get a structured signal: a mean prediction and a variance, a posterior over weights, or a per-input score that says how unusual this input looks compared to what the model was trained on. That signal is what powers the four capabilities above, and the rest of this chapter is the toolkit for producing it.

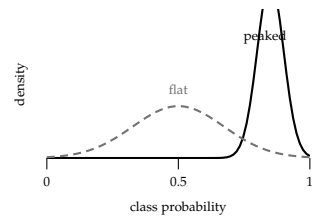

Figure 2: Two prediction profiles for the same kind of input. Solid: peaked, no uncertainty signal. Dashed: flat, high epistemic uncertainty. Both predict the same top class, but only the second tells you the model is unsure.

THE LEARNING OBJECTIVES of this lecture:

- Distinguish aleatoric from epistemic uncertainty and explain why only the latter is reducible with more data.
- Apply MC Dropout and Deep Ensembles to quantify prediction uncertainty, and turn that uncertainty into anomaly and out-of-distribution scores.
- Design graceful degradation strategies that route uncertain predictions to human reviewers and abstain when the model has no business answering.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What is the difference between aleatoric and epistemic uncertainty, and which one can MC Dropout estimate?

- When would you prefer deep ensembles over MC Dropout, and what are the trade-offs in compute and quality?
- What distinguishes a point anomaly from a contextual one, and why does the distinction matter for method choice?
- Why does the Z-score fail on heavy-tailed or multimodal data, and what alternatives exist?
- When does k-NN distance fail in a way that LOF handles, and what is the underlying mechanism?
- Why might an autoencoder trained on normal data reconstruct an anomaly with low error, and how would you mitigate this?
- Why is high softmax confidence not a reliable indicator that an input is in-distribution, and what does the energy score see that softmax does not?
- How does distribution shift cause an uncertainty detector to silently degrade, and what are the operational signals that the detector is no longer trustworthy?
- Why should a deployed model include both a fail-safe default and a human-handoff mechanism, and how do you decide which branch is appropriate for a given uncertainty level?

RECAP of Key Concepts:

- Uncertainty comes in two forms: aleatoric, which is irreducible noise in the data, and epistemic, which reflects the model's ignorance and shrinks as more data arrives. Only the latter is actionable through more training data.
- MC Dropout and deep ensembles both quantify epistemic uncertainty by measuring disagreement across multiple predictions. MC Dropout is cheap to deploy on a single trained network; deep ensembles cost M training runs but produce more reliable uncertainty estimates on truly novel inputs.
- Scoring methods turn uncertainty into decisions through statistical, proximity, and reconstruction families, plus OOD-specific scores. The type of anomaly, point, contextual, or collective, dictates which family is appropriate, and the threshold is a cost decision rather than a statistical one.
- Energy-based OOD detection separates in- from out-of-distribution inputs more reliably than softmax confidence because energy is not normalised; the absolute scale of the logits carries information that the softmax discards.

- Every uncertainty method inherits its model of normal from the training data, and that data drifts. Calibration, distribution shift, and reconstruction blindness are predictable failure modes that operational monitoring is designed to catch before they cause silent failures.
- Graceful degradation composes confidence thresholds, fail-safe defaults, human handoff, and monitoring into a safety net. A deployed model that can refuse to answer the cases it should refuse on is more trustworthy than one that always answers.

EVERY UNCERTAINTY METHOD IN THIS CHAPTER inherits its model of normality from the training data, and that data covers only a sliver of the input space the deployed model will encounter. A fraud detector trained on last year's transaction patterns will keep waving through this year's novel scheme until enough damage accumulates to relabel the data and retrain. The detector still works as designed; it just no longer works for the world it now lives in, and there is nothing inside the detector itself that flags this, because the failure lives outside the feature space it was ever shown.

ROOT CAUSE. Every method in this chapter inherits its notion of normal from a narrow training distribution, and the long tail of novel inputs is structurally invisible to a narrowly-trained detector.

TRANSFER LEARNING addresses this brittleness by widening the feature space before the task ever arrives. Instead of training a narrow detector on a narrow dataset, we reuse a backbone that has already been pre-trained on web-scale data, so that the long tail of any one task already sits inside the head of the backbone's representation [2]. Foundation models such as CLIP, GPT, and SAM push this idea to the extreme: a single representation, learned once on hundreds of millions of image-text pairs or tokens, can be reused on classification, retrieval, or segmentation tasks it was never explicitly trained for, with zero or only a handful of labelled examples.

[2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021. https://arxiv.org/abs/2103.00020

THE NEXT CHAPTER EXPLORES how pre-trained vision-language models turn a text prompt into a zero-shot classifier, how few-shot fine-tuning, linear probing, and adapters reuse a frozen backbone on a new task, and what goes wrong when the source and target domains drift far enough apart that transfer becomes negative.

TEASER. How can a sentence like "a photo of a cat" act as a classifier for an image the model has never seen before?

FEEDBACK

Part III

Epistemic Pattern

Transfer Learning

The Flywheel of Generalized Embeddings.

The Why

In Chapter 6 we trained an encoder that mapped inputs into a structured latent space, where nearby points were similar and the bottleneck distilled what mattered. The space was useful, but it belonged to one model trained for one task. By the same logic, every new task would mean training that encoder again, from scratch, on a freshly labelled dataset.

Train once, reuse forever. Almost every modern vision and language system in production starts from someone else's pretrained encoder. The encoder is the expensive part, the task-specific head on top is cheap, and the choice of encoder determines almost everything that follows.

Real systems do not work this way. Almost no production model is trained from scratch. They start from a backbone someone else pretrained at scale, freeze or adapt it, and bolt a small task-specific head on top. The encoder is the asset, the head is a thin wrapper, and the same backbone is shared across many downstream tasks.

Labels are scarce. Collecting labelled data is the bottleneck for most tasks. A medical-imaging team might labour for months to label five hundred chest X-rays, while a competitive vision encoder needs millions of labelled images to train from scratch. The two scales do not meet.

Domains splinter. Every domain has its own vocabulary of inputs, satellite imagery, manufacturing defects, document layouts, X-ray modalities, and in principle each deserves its own encoder. In practice none of them can afford its own dataset, so the encoder has to come from somewhere else and adapt.

Compute is enormous. Pretraining a modern vision transformer or large language model costs hundreds of GPU-years. Few teams

can afford this once, none can afford it per task, so the cost of pre-training must be amortised across tasks rather than paid each time.

SOME CLASSES HAVE NO EXAMPLES AT ALL. Rare diseases, novel products, low-resource languages, things that have not yet happened, by definition cannot live in a labelled training set. A method that requires examples per class will never see them, and a system that depends on such a method will never serve them.

REUSE IS THE MECHANISM. A backbone trained at scale on a broad task supplies features that work, often surprisingly well, for tasks it was never trained on. Push the scale far enough and the backbone becomes a foundation model that handles new tasks without any task-specific training at all, because its embeddings already cover the world that the new task lives in.

IN ORDER TO MOVE FROM ONE-TASK-ONE-MODEL to many-tasks-one-backbone we have good reason to find ways to,

- reuse features learned at scale for downstream tasks with little or no labelled data.
- adapt pretrained representations efficiently, without paying full retraining cost.
- bridge modalities so that text descriptions can serve as classifiers for visual inputs.
- diagnose when transfer fails because source and target distributions are too far apart.

THE FUNDAMENTAL QUESTION this chapter answers is how a model trained on one corpus can serve a task it never saw, with little or no additional supervision, and how to pick the right adaptation strategy for the data and compute budget at hand.

Hands On Experience

CONSIDER that you have been handed a folder of two hundred holiday photos and asked to sort them into beach, mountain, city, and forest. You have no labels and no time to make any. Three months ago this would have meant a week of labelling and a from-scratch convolutional network on far too little data. Today it can be done in five lines of code, without labelling a single image.

- How does the model classify your photos if it never saw them?
- What does "knowing what a beach is" mean for an encoder?
- Why might the same trick fail if the four classes were "part A", "part B", "part C", and "part D" from a factory line?

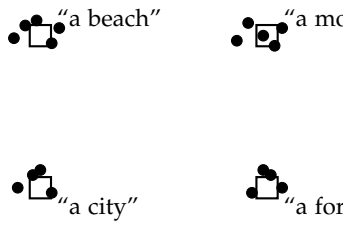

Figure 3: A shared embedding space. Image embeddings (filled dots) and text-prompt embeddings (open squares) live in the same space; the predicted class for each photo is the nearest prompt. The encoder was never trained on these specific photos or these specific class names.

The model never saw your photos. It was trained on hundreds of millions of image-caption pairs scraped from the web, none of them labelled with beach, mountain, city, or forest. Yet it can place your photos and your class names into a shared embedding space where similarity does the classification for you, because the words and images that produced its training data already covered most of what your photos contain.

From-scratch training would refuse this scenario. With two hundred unlabelled photos and four target classes, a network trained from scratch has no signal at all, and even with labels two hundred examples sit far below the threshold where a deep model finds anything that generalises. The clustering methods of Part I would group photos by colour and texture but cannot tell which group is the beach.

Reuse changes the question. The encoder has already done the heavy lifting on someone else's data. You inherit a feature space that knows what beaches and mountains look like, and your job shrinks to writing the right text prompt. The chapter develops the mechanism of reuse from this starting point, through the pretrained backbone you freeze and probe, to the foundation model you query in natural language.

The Learning Objectives of this lecture:

- Understand backbones, foundation models, and how they emerge from large-scale pretraining.
- Apply transfer strategies along a continuum, from frozen feature extraction and linear probing through fine-tuning, parameter-efficient adaptation, and prompting, and pick the right strategy by data regime and domain gap.
- Identify when zero-shot and few-shot approaches are appropriate, and diagnose when transfer fails because of distribution shift or negative transfer.

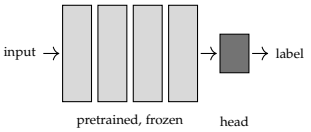

Figure 4: The signature operation of transfer learning: a pretrained backbone is frozen and a small head is trained on top. Most of the chapter is variations on what to freeze, what to adapt, and how cheaply.

Pretraining as infrastructure. Backbones are starting to look less like models you train and more like libraries you import. The encoder is built once at great cost and used many times at low cost, the same shape as compilers, operating systems, and standard libraries before it.

Embeddings, Backbones, Foundation Models

An embedding is a learned mapping from raw inputs to a vector space where geometry carries meaning. We met this idea in Chapter 6 with the autoencoder and the variational autoencoder. A high-dimensional input was compressed through a bottleneck, and the resulting low-dimensional code retained what the data had in common while discarding what it did not. The decoder was needed for training, but once training had finished, the encoder by itself was already a tool, a fixed function from inputs to a feature space where similarity, classification, and clustering all become easier than in the raw input.

The asset is the encoder. Everything else, the data loader, the loss function, the optimisation schedule, exists to produce one thing: an encoder whose embeddings are worth reusing.

What makes that encoder reusable is the bottleneck argument from Chapter 6. A k-dimensional embedding with $k \ll d$ cannot memorise its inputs. It has to throw away the noise and keep what generalises across examples, which is precisely the content that any downstream model needs in order to perform well on new inputs. The encoder that does this well for one corpus tends to do it well for many, and the rest of this chapter is the consequence.

Origin: From Hand-Crafted Features to ImageNet Pretraining

The reuse story did not start with deep learning. Computer vision has lived on shared feature extractors since the 2000s. SIFT, HOG, and their cousins were hand-engineered front ends that the entire field used in common, with task-specific classifiers bolted on top. The pattern was already there: build the encoder once, share it everywhere. What was missing was a way to learn the encoder from data instead of writing it down by hand.

Pre-2012 vision ran on hand-engineered features such as SIFT, HOG, SURF, and GIST. Each was a clever description of edges or gradients designed by humans. They worked, they were the state of the art for a decade, but they did not learn.

What changed in 2012 was the source of the encoder. AlexNet[3] was trained end-to-end on ImageNet and won that year's classification benchmark by a wide margin. Within two years it became clear that the network's penultimate layer was, in itself, a better feature extractor than any hand-engineered alternative for almost every other vision task. Razavian et al.[4] captured the lesson in a paper title that became a slogan: CNN features off-the-shelf, an astounding baseline. Yosinski et al.[5] quantified which layers transferred, finding that early layers were generic across tasks while late layers were tied to the source objective.

The lesson generalised quickly. Once the field had one ex-

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012. https://scholar.google.com/scholar?q=Krizhevsky+Sutskever+Hinton+2012+imagenet+classification+deep+convolutional+neural+networks

[4] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson. CNN features off-the-shelf: An astounding baseline for recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 806–813, 2014. https://arxiv.org/abs/1403.6382

[5] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014. https://arxiv.org/abs/1411.1792

ample of a network whose features were useful for everything else, it produced many more. ResNet, VGG, Inception, and later vision transformers[6] all became standard backbones, distributed as pretrained weights and reused by the rest of the field. Backbones became infrastructure in the same sense that compilers and operating systems are infrastructure: built once at great cost, used many times at low cost.

[6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.11929

Backbones, Defined

A BACKBONE is a pretrained encoder reused as a feature extractor. The two qualifiers matter. "Pretrained" means someone else paid for the training, on a different dataset, for a different objective. "Reused" means the encoder is treated as a fixed asset, plugged into a new pipeline whose only job is to produce the task-specific output on top.

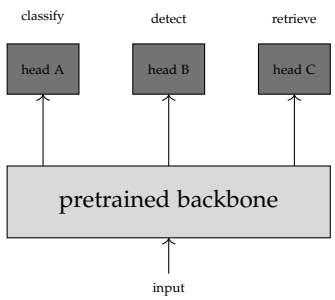

Figure 5: One backbone, many heads. The encoder is trained once on a broad pretraining task and serves as the input stage for many downstream tasks. The cost of the encoder is amortised across every task that shares it.

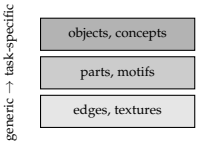

Figure 6: What each layer learns and how transferable it is. Early layers capture edges and textures generic across tasks; late layers capture concepts tied to the source task. The reuse strategies in Section 2 trade off these axes.

FIGURE 5 IS NOT A PIECE OF ARCHITECTURE, it is a piece of economics. Training the backbone is the expensive operation, paid for once. Each head is a small classifier or regressor that adapts the backbone's output to a specific task and is trained on a task-specific dataset that may be tiny. The backbone never sees the task-specific data while it is being built; the task-specific data never pays the cost of pretraining.

GENERALISATION is the reason this works. A bottlenecked encoder must compress, and compression requires ignoring idiosyncratic noise in favour of shared structure. A k-dimensional embedding with $k \ll d$ cannot memorise individual inputs; it must discover regularities that hold across the training distribution. Those regularities, the semantic features, are precisely what a downstream model needs to perform well on data it never saw during training.

Pretraining Datasets

WHERE THE BACKBONE COMES FROM is the design decision that matters most at this stage. The encoder is only as general as the data it was trained on, and across domains a small number of large-scale datasets have become the standard foundations for pretraining. Each engraves its own statistical fingerprint into every backbone built on top of it.

Dataset	Modality	Scale	What the backbone learns
ImageNet Deng et al. 2009	Images	14M images	Edges, textures, object parts, and category-level semantics. The de facto starting point for visual feature extractors.
The Pile Gao et al. 2020	Text	825 GB text	Syntax, coreference, factual associations, and discourse structure from books, academic papers, web text, and code.
LibriSpeech Panayotov et al. 2015	Audio	1000 h audio	Phoneme boundaries, prosody, and speaker-invariant representations from read English speech.
Kinetics-700 ao Carreira et al. 2018	Video	650k clips	Temporal dynamics, motion patterns, and action semantics from short video clips spanning 700 human action classes.
ShapeNet Chang et al. 2015	3D shapes	51k models	Surface geometry, part decomposition, and pose-invariant structure from annotated CAD models across 55 object categories.
LAION-5B Schuhmann et al. 2022	Image + text	5.8B pairs	Joint visual and linguistic semantics from web-crawled image-caption pairs; the foundation behind open multimodal models.
CheXpert Irvin et al. 2019	Medical images	224k X-rays	Anatomical landmarks, pathology signatures, and radiological texture patterns from labelled chest radiographs.
ZINC Sterling and Irwin 2015	Molecular graphs	250k molecules	Bond topology, functional-group motifs, and drug-likeness features from curated small-molecule structures.

Table 1: Large-scale pretraining datasets across modalities. Each dataset supplies the raw material for a backbone to learn general-purpose features that transfer to downstream tasks within and sometimes across its modality.

MODALITY MATTERS. An ImageNet backbone transfers well to satellite imagery or medical photos, but poorly to molecular graphs or audio spectrograms. Choosing the right source dataset and modality is the first design decision in any transfer learning pipeline.

Foundation Models

FOUNDATION MODELS are backbones at a different scale. The data is broader, the parameters are larger, and the training compute is larger still. The architecture is rarely new. What is new is what the back-

THE TERM was coined by the Stanford CRFM group . It is contested, but the phenomenon it names is real: a single pretrained model that serves dozens of downstream tasks without further training.

R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, et al. On

bone has seen and how much of it. CLIP[7] saw four hundred million image-caption pairs scraped from the web. GPT-style language models[8] saw most of the internet's text. SAM[9] saw a billion segmentation masks. The result is a backbone whose embeddings cover so much of the world that many tasks can be solved without any task-specific training, by querying the pretrained model directly.

The third ingredient, on top of scale and breadth, is alignment across modalities. CLIP's two encoders were trained jointly so that an image of a dog and the text "a photo of a dog" land near each other in a shared embedding space. The alignment is the bridge that turns a vision encoder into a zero-shot classifier in Section 3, and it is what separates a foundation model from an ordinary backbone in practice.

Notice that a foundation model is just a backbone with three additions: bigger, broader, and often cross-modal. There is no new architecture, no new layer type, no new optimiser. The leap is quantitative on every axis except one, and the qualitative consequence, emergent zero-shot capability, falls out of those quantitative differences. The remaining sections of this chapter work out how.

[7] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021. https://arxiv.org/abs/2103.00020

[8] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. https://arxiv.org/abs/2005.14165

[9] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Segment anything. *arXiv preprint arXiv:2304.02643*, 2023. https://arxiv.org/abs/2304.02643

The Transfer Continuum

SECTION 1 ASKED WHERE BACKBONES COME FROM. This section asks how to use one. The choice is not binary, freeze or retrain, but a continuum of techniques that trade two costs against each other: how much of the backbone you are willing to update, and how much labelled target data you are willing to spend updating it. At one extreme you change nothing and pay nothing in target labels. At the other you retrain the whole backbone and need a lot of target data. Most real projects sit somewhere between, and the rest of this section maps the space.

WHAT TO FREEZE, WHAT TO TRAIN. Every transfer technique answers the same two questions, which parameters move during adaptation, and how much target supervision is required to move them. Different answers sit at different points on a continuum.

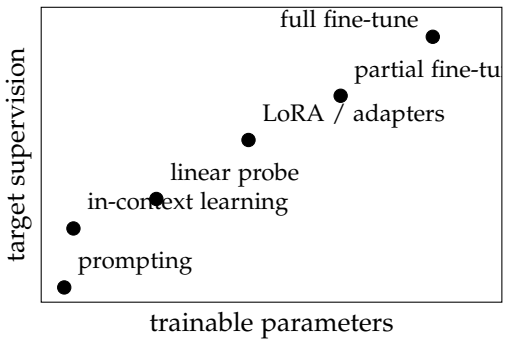

Figure 7: The transfer continuum. Each strategy occupies a position in the plane of trainable parameters against required target supervision. Cheaper strategies sit at the bottom-left and rely almost entirely on the backbone; more expensive strategies sit at the top-right and trade target labels for adaptability.

Frozen Features and Linear Probing

THE CHEAPEST REUSE STRATEGY is to freeze the backbone entirely and use it once. Feed every input through the encoder, store the resulting embeddings, and train a small classifier on those embeddings. The backbone is computed once per input, never updated, and the only training loop runs over the classifier on top.

LINEAR PROBING is the canonical version. Freeze the encoder f_θ and train a single linear layer W on top, optimising

$$\min_W \sum_i \mathcal{L}\big(W^\top f_\theta(x_i),\, y_i\big).$$

The objective has the same shape as logistic regression on hand-engineered features, except the features now come from a deep encoder. With a strong backbone, linear probing is often within a few points of full fine-tuning, at a small fraction of the cost.

LINEAR PROBING IS ALSO THE STANDARD DIAGNOSTIC for backbone quality. A new self-supervised method or a new pretraining

LINEAR-PROBE PROTOCOL. Compute embeddings once, cache them, sweep regularisation strengths on the cached vectors, pick by validation accuracy. The whole loop runs on a laptop even when the backbone needed a cluster to train.

dataset is almost always evaluated by reporting linear-probe accuracy on a fixed downstream benchmark, because the result isolates the encoder's contribution from the classifier's expressiveness. If the linear probe does well, the encoder is good; if it does badly, no amount of head engineering will rescue it.

Fine-Tuning

When source and target domains differ enough that frozen features are insufficient, the next step is to update some or all of the backbone's parameters. Fine-tuning unfreezes a chosen set of layers and trains them on the target data, typically with a much smaller learning rate than the original pretraining used.

Partial fine-tuning unfreezes only the late layers, leaving early layers untouched. This follows the layer-hierarchy logic of Section 1: early layers encode features that are generic across domains, so they rarely benefit from adaptation, while late layers encode task-specific concepts that often do. The learning-rate schedule frequently mirrors this, with later layers receiving larger updates than earlier ones.

Full fine-tuning unfreezes everything. It is the most flexible and the most expensive, both in compute and in target labels, and it carries the largest risk of catastrophic forgetting.

Catastrophic forgetting. A network that fine-tunes aggressively on a small target dataset can lose its ability to perform the source task. The encoder's earlier knowledge is overwritten by gradient updates that optimise only the new objective. Lower learning rates, fewer epochs, layer-wise learning-rate decay, and replay of source examples all reduce the damage.

Parameter-Efficient Fine-Tuning

Full fine-tuning becomes impractical once a backbone has billions of parameters. Storing one optimiser state per parameter, updating each weight, and saving a separate checkpoint per downstream task all scale with model size, and at modern scale none of them is cheap. Parameter-efficient fine-tuning, or PEFT, freezes the backbone and adds a small number of trainable parameters elsewhere, in such a way that the original weights are recovered when the new parameters are zero.

LoRA[10] is the dominant PEFT technique. Leave the backbone weights W frozen and represent the update ΔW as a low-rank matrix product:

$$W_{\text{adapted}} = W + \Delta W,$$
$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ with $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Only A and B are trained, so the trainable parameter count drops from

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685

$d_{\text{in}}d_{\text{out}}$ to $r(d_{\text{in}}+d_{\text{out}})$. For a 7B-parameter language model and rank $r=8$, the LoRA adapters add a few tens of megabytes against a backbone that occupies tens of gigabytes.

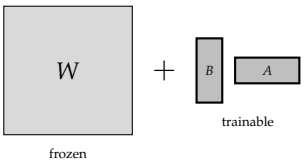

Figure 8: LoRA freezes the original weight matrix W and adds a low-rank update BA. The product has the same shape as W but only $r(d_{\text{in}}+d_{\text{out}})$ trainable parameters.

THE DEEPER CONSEQUENCE is that adaptation becomes composable. A single frozen backbone can serve many tasks, each with its own small LoRA adapter swapped in at inference time. The economics of Section 1 reappears: the backbone is the asset, the adapters are cheap task-specific extensions, and many of them coexist on top of one shared encoder.

PEFT SIBLINGS. Adapters insert small bottleneck layers between transformer blocks. BitFit fine-tunes only the bias terms. Prompt tuning learns a soft prompt of trainable token embeddings prepended to the input. Each occupies a different point on the parameter-cost-versus-flexibility trade-off, and LoRA happens to be the most popular because it preserves inference latency.

N. Houlsby, A. Giurgiu, S. Jastrzębski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2790–2799, 2019. https://arxiv.org/abs/1902.00751; and E. B. Zaken, Y. Goldberg, and S. Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1–9, 2022. https://arxiv.org/abs/2106.10199

Prompting and In-Context Learning

THE EXTREME PARAMETER-EFFICIENT CASE is to train no parameters at all. Some foundation models, especially large language models, can be steered by the input alone, without any gradient updates. The control surface is the prompt, the text or other input that precedes the actual query and conditions the model's behaviour on it.

ZERO-SHOT PROMPTING sends only the task description, for example, "Classify the following review as positive or negative. Review: . . . ". In-context learning sends a handful of input-output pairs alongside the description, before the actual query, and the model imitates the pattern. No weights move, no gradients flow, and yet the output adapts to the examples it just saw.

PROMPTING IS PARAMETER-FREE but it is not free. Each example added to the prompt increases the input length, which costs compute and latency at inference, and the prompt has to fit within the model's context window. Prompt engineering, the discipline of writing prompts that get the desired behaviour, becomes a real engineering practice, with template libraries, ablation studies, and structured outputs as standard tools.

ICL DETAILS belong to the upcoming lecture series on large language models. Here ICL is just the no-gradient endpoint of the transfer continuum, the smallest possible parameter cost, paid in prompt tokens instead of weight updates.

Choosing a Strategy

THE CONTINUUM YIELDS A RECIPE. Pick the cheapest strategy that meets the target accuracy, and only pay for more parameters when the cheaper option provably falls short.

RECIPE. The table below is a starting point, not a law. Real projects iterate, often starting at the cheap end and moving rightwards on the continuum only when the cheaper option is shown to be insufficient.

NOTICE THAT THE ORDERING is by parameter cost, not by accuracy. A linear probe on a strong backbone often beats full fine-tuning on a weak one, especially when target data is scarce. The right question is

Target labels	Domain gap	Strategy	Why
Zero	Small	Zero-shot prompting	The backbone already covers the target task; describe the task in text and read the answer.
A few examples	Small	In-context learning	Show the model the pattern in the prompt; no training needed.
Tens to hundreds	Small	Linear probe	Frozen embeddings plus a single linear layer, cheap and reproducible.
Hundreds to thousands	Medium	LoRA / adapter	Update a small number of parameters and preserve the backbone for other tasks.
Thousands or more	Medium to large	Partial fine-tuning	Adapt the late layers where task specificity lives.
Many thousands or more	Large	Full fine-tuning	Update everything, accept the catastrophic-forgetting risk, and budget for compute.

Table 2: A starting recipe for choosing a transfer strategy. Move down the table as the domain gap and the volume of available target labels grow.

rarely which strategy is most powerful, but which strategy makes the best use of the labels and compute on hand.

When Transfer Fails

Failure mode. Transfer is not a free lunch. The backbone supplies a strong inductive bias, and when that bias is wrong for the target task, transfer hurts.

Negative transfer is the case where reusing a pretrained backbone produces lower accuracy than training from scratch would. It happens when the source and target distributions are far enough apart that the backbone's features are actively misleading rather than merely uninformative. An ImageNet backbone applied to molecular graphs, a language model fine-tuned on legal documents that drifts toward casual web tone, or a satellite-imagery encoder pulled into medical X-rays all illustrate the same failure: the backbone's prior fights the target objective instead of helping it.

Catastrophic forgetting is the related failure of fine-tuning. When the model is updated on the target task with a learning rate that is too aggressive, or for too many epochs, the original capabilities are overwritten by gradient updates whose only signal is the new objective. The model becomes good at the target task and bad at everything else, which is a problem when "everything else" was the reason the backbone was useful in the first place.

Distribution shift is the operational version of negative trans-

fer. The model was trained on one distribution and is deployed on another, and the gap between the two erodes performance silently. Three flavours recur. Covariate shift occurs when $P(X)$ changes but $P(Y \mid X)$ stays the same: the inputs look different but the labelling rule is unchanged. Label shift occurs when $P(Y)$ changes but $P(X \mid Y)$ stays the same: class proportions move but each class still produces the same kind of inputs. Concept drift occurs when $P(Y \mid X)$ changes over time: the same input now carries a different label, often because the world has changed.

THE REMEDIES FOR EACH SHIFT TYPE differ. Covariate shift is often addressed by importance weighting or by domain adaptation. Label shift can be corrected analytically when the new class proportions are known. Concept drift typically requires retraining, with the harder challenge of detecting the drift before the model is already wrong. Chapter 7 supplies the operational tools, calibration, OOD detection, and monitoring, that catch these failures in deployment, and this chapter supplies the adaptation techniques that respond to them.

ADVERSARIAL DOMAIN ADAPTATION learns features that are informative for the task but indistinguishable across domains, by training a domain discriminator and asking the encoder to fool it. The technique is a clean instance of the broader pattern, learn invariances explicitly when the data does not supply them.

Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky. Domain-adversarial training of neural networks. In *Journal of Machine Learning Research*, volume 17, pages 1–35, 2016. https://arxiv.org/abs/1505.07818

Zero- and Few-Shot Learning

SECTION 2 ENDED WITH A RECIPE that ranges from full fine-tuning to prompting, ordered by how much of the backbone is allowed to move. The cheap end of that continuum, prompting and in-context learning, is also the regime where target labels are scarcest. This section follows the cheap end into its sharpest form: classifying with no labelled examples at all, or with a handful, by leaning entirely on a backbone that already knows enough about the world to recognise the new task when it sees it.

ZERO EXAMPLES, ONE EXAMPLE, A HANDFUL. Section 2 mapped strategies by how many parameters they update. Here we map them by how many target-class examples they need. The two axes meet at the bottom-left corner: zero parameters and zero examples, the regime that used to be impossible.

Origin: Attribute-Based Zero-Shot

THE FIRST FORMULATION OF ZERO-SHOT classification in computer vision came from Lampert and colleagues in 2009[11]. They asked a simple question: can a model classify an animal it has never seen during training, given only a description of what that animal looks like? Their answer was a hand-built class-attribute table. Each class, seen or unseen, was encoded as a vector of binary attributes such as striped, four-legged, mammal, lives-in-water. A zebra became striped + four-legged + mammal + black-and-white; a dolphin became smooth + lives-in-water + mammal.

THE BRIDGE PROBLEM. Zero-shot classification needs some link between class semantics and image features. Without examples of the new class, the link cannot be learned from data; it has to come from somewhere else. Pre-2020 it came from a hand-built table.

[11] C. H. Lampert, H. Nickisch, and S. Harmeling. Learning to detect unseen object classes by between-class attribute transfer. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 951–958, 2009. `https://scholar.google.com/scholar?q=Lampert+Nickisch+Harmeling+2009+learning+detect+unseen+object+classes+between-class+attribute+transfer`

THE MODEL WAS NOT TRAINED ON CLASSES, it was trained on attributes. Each attribute had its own classifier, trained on whichever seen classes had that attribute. At inference time, an image was passed through every attribute classifier, the outputs were assembled into a predicted attribute vector, and the image was assigned to the class whose attribute vector lay closest. The unseen classes had never produced a single training image, but their attribute descriptions slotted into the same table as the seen classes, so the comparison was well defined.

THE LESSON OUTLASTED THE METHOD. Zero-shot classification needs a bridge between class semantics and image features, and the bridge has to come from outside the labelled training set. Lampert's bridge was a hand-curated attribute ontology, which is expensive to build, narrow in coverage, and hard to extend. The structural insight, that an external semantic representation of the target classes is what makes zero-shot possible, survived long after the table-of-attributes implementation was abandoned.

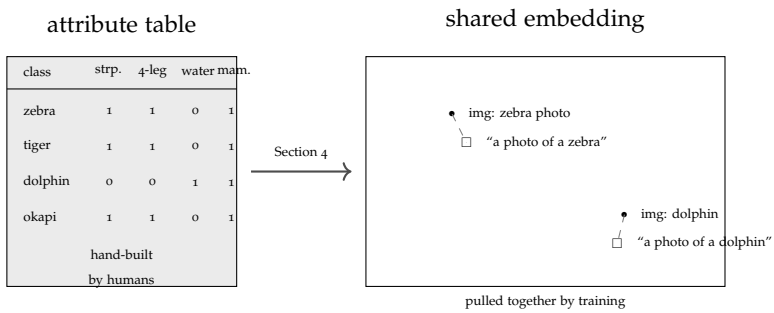

Figure 9: Two ways to bridge class semantics and image features. On the left, Lampert's attribute table encodes each class as a vector of human-curated attributes; the model classifies images by predicting attributes and matching against the table. On the right, a cross-modal pretraining objective places images and class-name texts into a shared embedding space, so the bridge is built from data instead of by hand.

The Cross-Modal Leap

The bridge that Lampert built by hand can be built from data, if the data already contains both modalities. The web is full of image-caption pairs, and a model trained to align images with their captions ends up learning, as a side effect, an embedding space where the text "a photo of a zebra" and an actual photo of a zebra land near each other. The attribute table is replaced by a text encoder, and the bridge becomes a learned function rather than an authored ontology.

Zero-shot classification then collapses to nearest-prompt similarity. For a target task with classes $\{c_1, \ldots, c_K\}$, each class name is wrapped in a prompt template, embedded by the text encoder, and the predicted class for an image x is

$$\hat{y} = \arg\max_k \cos\big(f_{\text{img}}(x),\, f_{\text{txt}}(T(c_k))\big),$$

where T is a template such as "a photo of a {class}". There is no training loop, no gradient, no labelled image of the target classes. The classifier is a similarity computation in a space that the backbone built once, on someone else's data.

Section 4 will derive this picture in detail for CLIP, including the contrastive objective that produces the shared space and the diagonal-loss training pattern. For now the picture is enough: the cross-modal pretraining of Section 4 supplies, for free, exactly the bridge that Lampert had to author.

Few-Shot Recipes in the Embedding Space

When a few labelled examples per class are available, the same backbone supports a small family of recipes that share a common ingredient: every example, labelled or query, is mapped

through the frozen encoder once, and the few-shot logic happens entirely in the embedding space.

THE LINEAR PROBE of Section 2 is the most direct option. Given K shots per class, embed each example with the frozen backbone, and fit a single linear layer on the cached embeddings. With strong backbones and modest K this is often a few accuracy points off full fine-tuning, runs on a laptop, and never touches the backbone weights.

PROTOTYPICAL NETWORKS[12] replace the linear classifier with a centroid. For each class k, compute the mean of the few labelled embeddings,

$$\mu_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} f_\theta(x_i),$$

and classify a query x by nearest centroid in the embedding space,

$$\hat{y} = \arg\min_k \|f_\theta(x) - \mu_k\|.$$

There are no parameters to fit beyond the centroids themselves, and the centroids are computed from the support set in closed form. Nearest-class-mean classification, the same computation under a different name, predates the deep learning era and is the canonical baseline whenever a strong embedding is on hand.

[12] J. Snell, K. Swersky, and R. S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://arxiv.org/abs/1703.05175

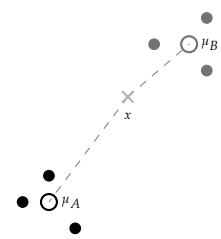

Figure 10: A prototypical-networks classifier in the embedding space. Three labelled examples per class produce two prototypes μ_A and μ_B as their centroids. The query x is classified by the closer prototype, here μ_B.

NOTICE THAT ALL THREE RECIPES SHARE ONE INGREDIENT, the encoder of Section 1 used as a feature extractor. The linear probe, the prototypical network, and the nearest-class-mean classifier differ only in what they do with the resulting embeddings. The heavy lifting is upstream, in the pretraining, and the few-shot stage merely chooses how to convert support embeddings into a decision rule.

Prompt Engineering

THE PROMPT IS A HYPERPARAMETER. A zero-shot classifier built from a foundation model is sensitive to how the class names are wrapped in text, because the text encoder produces different embeddings for "zebra", "a photo of a zebra", and "a photo of a zebra, a striped african equid". The first reads as a bare word, the second as a typical caption, and the third supplies the kind of disambiguating context that captions in the training data tended to carry. The third tends to win.

PROMPT ENSEMBLING. Averaging the text embeddings of dozens of paraphrased prompts smooths over the idiosyncrasy of any single template. The trick is mechanical, the gain is real, and on ImageNet zero-shot it is worth several accuracy points.

HAND-CRAFTED TEMPLATES dominate the simplest practice. "A photo of a {class}" became the de facto baseline because it matches

the caption distribution that web-trained models saw most often. Domain-specific templates beat the generic baseline whenever the target task has a recognisable register: "a satellite photo of {class}" for remote sensing, "a histopathology slide of {class}" for medical imaging, "a sketch of {class}" for line art.

CoOp[13] replaces the human-written template with learnable token embeddings. The class name is preceded by M continuous vectors $V_1, \ldots, V_M$, optimised on a small labelled set so that the resulting prompt embedding pulls the right images closest. The vectors live in the embedding space, not in the text vocabulary, so the learned "prompt" is no longer a sentence; it is a soft prefix that the text encoder reads instead of human language. CoOp routinely beats the best hand-crafted template at the cost of a tiny labelled set and a short training run, and sits squarely in the PEFT corner of the Section 2 continuum.

[13] K. Zhou, J. Yang, C. C. Loy, and Z. Liu. Learning to prompt for vision-language models. In *International Journal of Computer Vision*, volume 130, pages 2337–2348, 2022. https://arxiv.org/abs/2109.01134

From Prompts to Aligned Models: A Thumbnail

THE PROMPTING STORY DOES NOT END WITH VISION-LANGUAGE MODELS. Large language models walked their own path from raw prompting to aligned behaviour, and the milestones along the way explain why the prompts a modern student types into a chat interface look almost nothing like what GPT-3 expected. Each step kept the previous one and added a control surface on top.

THUMBNAIL ONLY. The full story of large language models, alignment, and reinforcement learning from human feedback belongs to the upcoming LLM lecture series. Here we trace the names of the steps and a one-line framing of each, so that the prompt-engineering beat above lands in its proper place in the timeline.

RAW FEW-SHOT PROMPTING was the GPT-3 demonstration[14]. A model trained only to predict the next token, with no notion of tasks or instructions, was shown a handful of input-output examples in the prompt and continued the pattern for a new input. Chain-of-thought prompting[15] added a single rhetorical move, asking the model to think step by step, and unlocked multi-step reasoning that direct prompting could not elicit. Soft prompting and prefix tuning[16] replaced the human-written prefix with a small set of trainable embeddings, the language analogue of CoOp, and showed that the prompt could itself be learned at a tiny parameter cost. Instruction tuning[17] fine-tuned the model on collections of (instruction, response) pairs covering many tasks, so that natural-language instructions, rather than few-shot examples, became the primary control surface. Reinforcement learning from human preferences[18] provided the next layer: humans ranked candidate outputs, a reward model fit those rankings, and the language model was nudged towards preferred behaviour by policy optimisation; InstructGPT[19] brought the recipe to

[14] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. https://arxiv.org/abs/2005.14165

[15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022b. https://arxiv.org/abs/2201.11903

[16] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059, 2021. https://arxiv.org/abs/2104.08691; and X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4582–4597, 2021. https://arxiv.org/abs/2101.00190

[17] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned lan-

language models at scale and produced the first chat-style assistants. Direct preference optimisation[20] simplified the picture by removing the explicit reward model, treating the language model itself as the reward, and optimising preference data with a single supervised objective.

[20] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. https://arxiv.org/abs/2305.18290

The thread through these steps is one that this section has already walked in miniature. Each milestone moves the locus of control from the model's weights, to the prompt, and then back into the weights through ever-cheaper supervision: examples in context, learned soft prefixes, instruction-response pairs, ranked preferences, direct preference data. The names matter because they carry the field's vocabulary; the mechanism matters because each one is a way of nudging a backbone whose weights cost too much to retrain. Section 4 returns to vision-language alignment with the contrastive objective that built the embedding space this whole section has stood on.

CLIP and Cross-Modal Alignment [placeholder]

PLACEHOLDER. Section 4 will be drafted by a section-drafter subagent. Scope: contrastive image-text objective, diagonal-loss training figure, one worked zero-shot classification trace; CLIP siblings (SigLIP/ALIGN/BLIP) collapsed into ONE marginnote; multimodal LLMs/VLMs in a separate marginnote pointing at the LLM lecture series.

Composing Foundation Models [placeholder]

PLACEHOLDER. Section 5 will be drafted by a section-drafter subagent. Scope: RAG (retrieval-augmented generation) as the canonical composition centerpiece, an embedding backbone (callback to §1) plus a frozen generator; CLIP+SAM+LLM as a secondary multimodal composition; ch14 VLA teaser marginnote. RAG is the use-payoff that closes the algorithmic arc.

Self-Reflection and Recap

SELF-REFLECTION.

1. Why does CLIP generalise to unseen categories?
2. When does fine-tuning outperform zero-shot, and when is the reverse true?
3. How does prompt ensembling reduce the sensitivity to a single prompt choice?
4. What makes CoOp's learned prompts more effective than hand-crafted ones?
5. When would you prefer a linear probe over full fine-tuning for few-shot tasks?
6. How does adversarial domain adaptation differ from simply fine-tuning on target data?
7. What risks arise when composing multiple foundation models in a pipeline?

RECAP.

- Transfer learning reuses pre-trained features; early layers transfer well, but late layers require adaptation, and large domain gaps risk negative transfer.
- Zero-shot classification via CLIP matches image embeddings to text prompt embeddings; prompt design and ensembling are critical for accuracy.
- Few-shot strategies range from linear probing to full fine-tuning; the right choice depends on how much labelled data is available and how far the target domain is from the source.
- Domain adaptation techniques learn features that are informative for the task but invariant to the domain, mitigating covariate shift.
- Foundation models such as CLIP, SAM, and DINO serve as composable building blocks for tasks they were never explicitly trained on.

THE BACKBONE ARRIVES READY-MADE. Throughout this chapter we have taken pretrained encoders as given and asked how to reuse them. ImageNet supervised pretraining works for natural images because someone else once labelled fourteen million examples, but no second ImageNet is being built for every new domain. Foundation models such as CLIP escape this bottleneck not by labelling more, but by extracting supervision from data that was already there, in the form of image-caption pairs scraped from the web. The supervision was hiding in plain sight.

ROOT CAUSE. Every method in this chapter assumes the backbone already exists. Pretraining is treated as someone else's problem; the next chapter shows whose problem it is and how it gets solved without labels.

SELF-SUPERVISED LEARNING generalises this idea. Instead of asking humans for labels, the model is given a task that can be solved using only the structure of the data itself: predict the masked patch from the surrounding patches, pull augmented views of the same image close together, predict the next token from the previous ones. The labels come from the data, and the representations that emerge are the very backbones this chapter has taught us to reuse.

TEASER. If we already have the data, where in the data do the labels hide?

THE NEXT CHAPTER EXPLORES how data structure becomes its own supervision signal, walking through the contrastive and predictive families of self-supervised objectives that produced the foundation models we have just learned to reuse.

FEEDBACK

Part IV

Aleatoric Pattern

Index

aleatoric uncertainty, 15
Anomaly Detection, 15
attribute-based zero-shot, 37

backbone, 25, 28, 29

catastrophic forgetting, 33
CLIP, 25, 42
composition, 43
concept drift, 36
CoOp, 40
covariate shift, 36
cross-modal alignment, 38, 42

distribution shift, 16, 36
DPO, 40

embedding, 28
epistemic uncertainty, 15
explainable AI, 16

few-shot learning, 37, 38
fine-tuning, 32, 33
foundation model, 25, 26, 28, 30

generalisation, 29

ICL, 34
in-context learning, 34
instruction tuning, 40

label shift, 36
license, 2
linear probe, 32
LoRA, 33

negative transfer, 35

one-class learning, 17
OOD Detection, 15
operational design domain, 16

PEFT, 33
prompt engineering, 34, 39
prototypical networks, 38

recap, 19
reflection, 19
retrieval-augmented generation, 43
RLHF, 40

Transfer Learning, 25
transfer learning, 32

Uncertainty Estimation, 15
uncertainty estimation, 15

Zero-Shot, 25
zero-shot learning, 37